

RECOPILACION DE PROPIEDADES Y METODOS DEL DOM

Como ya sabes si quieres conocer propiedades y métodos de los elementos DOM, así como su uso podemos acceder a http://www.w3schools.com/jsref/dom_obj_all.asp

En este apartado haremos un repaso de alguna de ellas que podrás utilizar.

a) Propiedades relacionadas con la **estructura** del DOM.

Son propiedades que guardan relación con el **parentesco** del nodo que hemos **seleccionado** y se **utilizan** para **seleccionar** a su vez **otro nodo** a partir de este. Por ejemplo, es como si yo tuviera una propiedad que es hermanos, cuyo valor sería mi hermano Uri. Las propiedades más empleadas de este tipo son:

- Node.**childNodes**: los nodos hijos.
- Node.**firstChild**: el primer hijo.
- Node.**lastChild**: el último.
- Node.**nextSibling**: el primer hermano. (Sibling= Hermana/o)
- Node.**previousSibling**: el hermano anterior.
- Node.**parentElement**: el padre del tipo elemento.
- Node.**parentNode**: el nodo padre, sea cual sea.

Por ejemplo, así añadiríamos una clase al ul padre de un li.

```
<ul>

<li id="itemLista">Soy un nodo hijo</li>

<li>Soy el hermano del anterior</li>

</ul>

<script>

var miNodo = document.getElementById('itemLista');

miNodo.parentNode.setAttribute('class',
'claseAñadida');

</script>
```

b) Propiedades relacionadas la **naturaleza** del nodo.

Como ya vimos, podemos saber qué tipo de nodo es cada cual con alguna de estas propiedades:

- Node.**nodeName**: el **nombre** del nodo.
- Node.**nodeType**: el **tipo** de nodo.
- Element.**tagName**: la **etiqueta**.

c) Propiedades relacionadas con el **contenido** del nodo.

- Node.**textContent**: la cadena de texto que contiene el nodo.
- Element.**innerHTML**: el contenido, incluido los tags html, del nodo.

Por ejemplo, así cambiaríamos el contenido de texto de un nodo:

```
<div id="texto">Soy el contenido original</div>

<script>

var miNodo = document.getElementById('texto');

miNodo.textContent = "Cambiado";

</script>

Y así si quisiéramos incluir tags de html:

miNodo.innerHTML = "<h1>Cambiado</h1>";
```

d) Propiedades relacionadas con el **tamaño** y la posición del nodo

- Element.**clientHeight**: el alto.
- Element.**clientWidth**: el ancho.
- Element.**clientLeft**: su **distancia** respecto al margen **izquierdo** del navegador.
- Element.**clientTop**: su **distancia** con el margen **superior**.
- Element.**offset**: su posición **x e y**.

e) Propiedades y métodos relacionados con los atributos y estilos de un elemento.

- **Element.attributes**: una colección con **todos los atributos** del elemento.
- **Element.className**: el nombre de las clases del elemento.
- **Element.id**: el id.
- **Element.name**: el atributo name en los tags que pueden llevarlo, como los relacionados con los formularios o los enlaces.
- **Element.setAttribute()**: **cambia** el valor de un atributo.
- **Element.getAttribute()**: **selecciona** un atributo.
- **Element.hasAttribute()**: **comprueba** si tiene un atributo.
- **Element.removeAttribute()**: remueve un atributo.
- **Element.style**: para modificar algunos estilos en línea. Puedes ver los estilos disponible en <https://developer.mozilla.org/es/docs/DOM/CSS>

f) Métodos relacionados con la inserción y eliminación de nodos.

Listados así, los métodos siguientes puede que nos suenen un poco a chino, pero en la segunda parte de esta entrada nos quedarán más claros.

- **Node.appendChild()**: **inserta** un elemento **al final** de los contenidos del elemento padre en el que se está incluyendo.
- **Node.insertBefore()**: justo al revés, lo **inserta** el primero.
- **Node.removeChild()**: **suprime** un elemento.
- **Node.replaceChild()**: lo **reemplaza**.
- **Node.cloneNode()**: **copia** un elemento que luego podemos insertar mediante algún método.
- **Element.insertAdjacentHTML()**: **inserta** un fragmento de HTML antes o después de un nodo. *
- **document.createElement()**: **crea** un elemento, **que luego se inserta** en el DOM con alguno de los **métodos** anteriores.

*** insertAdjacentHTML():**

<https://developer.mozilla.org/es/docs/Web/API/Element/insertAdjacentHTML>

```
element.insertAdjacentHTML(posición, texto);
```

posición

Un **DOMString** que representa la posición relativa al elemento, y deberá ser una de las siguientes cadenas:

- **'beforebegin'**: Antes que el propio elemento.
- **'afterbegin'**: Justo dentro del elemento, antes de su primer elemento hijo.
- **'beforeend'**: Justo dentro del elemento, después de su último elemento hijo.
- **'afterend'**: Después del propio elemento.

texto

Es la cadena a ser analizada como HTML o XML e insertada en el árbol.

Hay varias formas principales de andar **quitando y añadiendo** nodos.

Una es con el **innerHTML** que vimos antes, **otra** es con la **combinación** de un **createElement()** o un **cloneNode()** y luego un **appendChild()** o un **insertBefore()**. Por ejemplo, así insertaríamos un **** al final de un listado.

```
<ul id="soyUnaLista">

<li>Item lista</li>

<li>Item lista 2</li>

</ul>

<script>

var miNuevoItem = document.createElement('li');

miNuevoItem.textContent = "Item lista nuevo";

var lista = document.getElementById('soyUnaLista');

lista.insertBefore(miNuevoItem);

</script>
```

g) Métodos relacionados con la **selección de nodos**.

- `document.getElementById('id')`.
- `document.querySelector('selector')`.
<https://developer.mozilla.org/es/docs/Web/API/Element/querySelector>
→ devuelve el **primer descendiente** del elemento sobre el cual es invocado que coincida con el o los selectores especificados.
- `document.getElementsByTagName('nombreTag')`.
- `document.getElementsByClassName('nombreClase')`.
- `document.querySelectorAll('selector')`.
<https://developer.mozilla.org/es/docs/Web/API/Document/querySelectorAll>
→ devuelve una `NodeList` estática (no viva) que representa una **lista de elementos** del documento que **coinciden** con el grupo de **selectores** indicados.

Selectors:

Un `DOMString` que contiene uno o más selectores para buscar coincidencias. Esta cadena de texto debe ser una cadena [CSS selector](#) válida; si no lo es, se lanzará una excepción `SyntaxError`.

* `NodeList`: son colecciones de nodos como los devueltos por propiedades como `Node.childNodes` y el método `document.querySelectorAll()`.

<https://developer.mozilla.org/es/docs/Web/API/NodeList>

No se han listado todos los métodos y propiedades, pero **sí las más importantes** en lo que nos atañe, el trabajo del DOM.

Más info en : https://www.w3schools.com/jsref/dom_obj_all.asp

insertBefore() Method

`node.insertBefore(newnode, existingnode)`

<i>newnode</i>	Required. The node object you want to insert
<i>existingnode</i>	Required. The child node you want to insert the new node before. If set to <code>null</code> , the <code>insertBefore</code> method will insert the <i>newnode</i> at the end

Example

Insert a new `<li>` element before the first child element of an `<ul>` element:

```
var newItem = document.createElement("LI"); // Create a <li> node
var textnode = document.createTextNode("Water"); // Create a text node
newItem.appendChild(textnode); // Append the text to <li>
```

```
var list = document.getElementById("myList");// Get the <ul> element to insert a new node
list.insertBefore(newItem, list.childNodes[0]);// Insert <li> before the first child of
<ul>
```